

Spring Boot

Или как начать пользоваться спрингом
используя best-practice и реализации по-умолчанию

Spring Boot

Дополнение к Spring, которое облегчает и ускоряет работу с ним. Сам Spring Boot представляет собой набор утилит, автоматизирующих настройки фреймворка.

Сам по себе Spring имеет достаточно большое количество зависимостей и множество сложных конфигураций, которые к тому же чаще всего описаны в XML, из-за чего у рядового разработчика могут возникнуть проблемы при знакомстве с фреймворком. Для упрощения этого процесса на помощь приходит Spring Boot — самый быстрый и популярный способ запуска Spring-проектов.

Типовые шаги для старта без Spring Boot

Каждый раз, создавая очередное корпоративное Java-приложение на основе Spring, вам необходимо повторять одни и те же рутинные шаги по его настройке:

- В зависимости от типа создаваемого приложения (Spring MVC, Spring JDBC, Spring ORM и т. д.) импортировать необходимые Spring-модули
- Импортировать библиотеку web-контейнеров (в случае web-приложений)
- Импортировать необходимые сторонние библиотеки (например, Hibernate, Jackson), при этом вы должны искать версии, совместимые с указанной версией Spring
- Конфигурировать компоненты DAO, такие, как: источники данных, управление транзакциями и т. д.
- Определить класс, который загрузит все необходимые конфигурации



Основные цели Spring Boot

- Обеспечить быстрый и широко доступный опыт начальной работы для любых разработок на Spring.
- Возможность кастомизировать стандартное поведение.
- Предоставлять ряд нефункциональных возможностей, которые являются общими для больших классов проектов (таких как встроенные серверы, безопасность, метрики, проверка работоспособности, тестирование и конфигурация).
- Уйти от старых подходов с XML-конфигурациями

За счёт чего достигаются эти цели?

Согласно <https://spring.io/projects/spring-boot>:

- 'starter' зависимости с облегчающие конфигурацию
- Автоматическая конфигурация различных библиотек
- Преднастроенный Application Server (Apache Tomcat)
- Готовые рецепты для широко используемых подходов (таких как метрики, внешняя конфигурация и т.д.)

Starter packages

Всё удобство **Spring Boot** основано на использовании так называемых *Starter*, которые позволяют получить набор сконфигурированных бинов, готовых к использованию и доступных для конфигурации через properties-файлы.

Краеугольным камнем инфраструктуры **Spring Boot** являются *AutoConfiguration*-классы, которые **Spring Boot** находит при запуске приложения и использует для автоматического создания и конфигурирования бинов.

Starter packages

Starter-пакеты представляют собой набор удобных дескрипторов зависимостей, которые можно включить в свое приложение. Это позволит получить универсальное решение для всех, связанных со Spring технологий, избавляя программиста от лишнего поиска примеров кода и загрузки из них требуемых дескрипторов зависимостей.

Например, если вы хотите начать использовать Spring Data JPA для доступа к базе данных, просто включите в свой проект зависимость **spring-boot-starter-data-jpa** и все будет готово (вам не придется искать совместимые драйверы баз данных и библиотеки Hibernate).

Starter packages

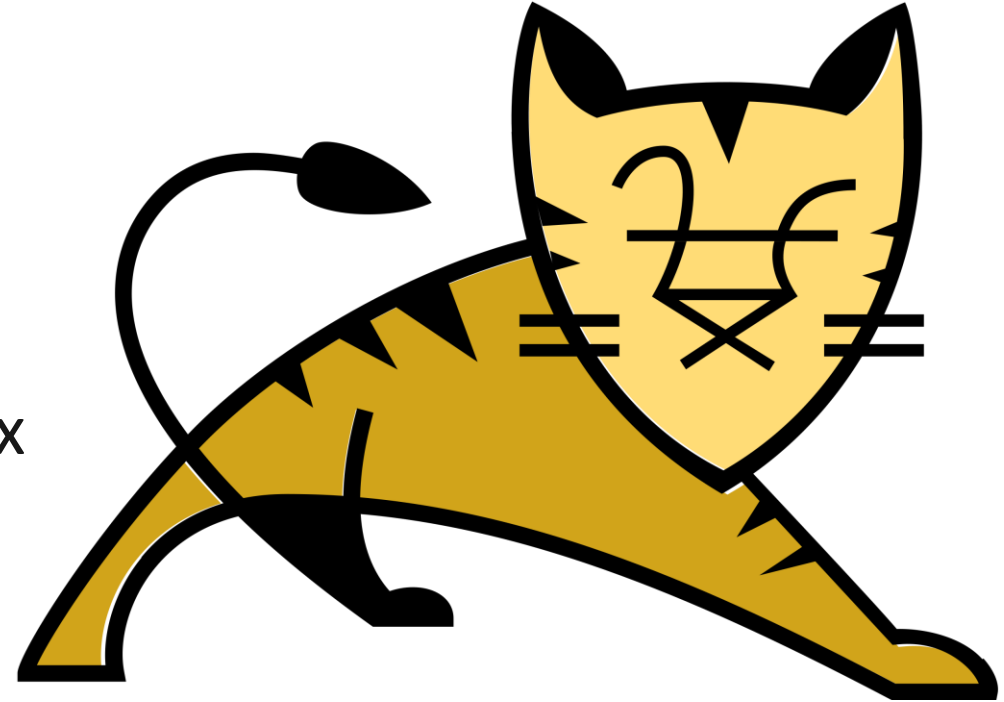
Также, если вы хотите создать Spring web-приложение, просто добавьте зависимость **spring-boot-starter-web**, которая подтянет в проект все библиотеки, необходимые для разработки Spring MVC-приложений, таких как **spring-webmvc**, **jackson-json**, **validation-api** и **Tomcat**.

Другими словами, **Spring Boot** собирает все общие зависимости и определяет их в одном месте, что позволяет разработчикам просто использовать их, вместо того, чтобы изобретать колесо каждый раз, когда они создают новое приложение.

Apache Tomcat

Apache Tomcat — это комплект серверных программ от Apache Software Foundation, предназначенный для тестирования, отладки и исполнения веб-приложений на основе Java. Его обычно называют контейнером сервлетов — дополнительных компонентов, которые расширяют функциональность веб-сервера и позволяют ему выполнять приложения на языке Java.

DispatcherServlet из Spring (с прошлой лекции) как раз крутится внутри этого самого Tomcat'a.



AutoConfiguration

Второй превосходной возможностью **Spring Boot** является автоматическая конфигурация приложения.

После выбора подходящего **starter**-пакета, **Spring Boot** попытается автоматически настроить Spring-приложение на основе добавленных вами **jar**-зависимостей.

Давайте рассмотрим на примере...

Java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
3  import org.springframework.web.bind.annotation.RequestMapping;
4  import org.springframework.web.bind.annotation.RestController;
5  @RestController
6  @EnableAutoConfiguration
7  public class MyApplication {
8      @RequestMapping("/")
9      String home() {
10         return "Hello World!";
11     }
12     public static void main(String[] args) {
13         SpringApplication.run(MyApplication.class, args);
14     }
15 }
```

AutoConfiguration

Вторая аннотация на уровне класса – это @EnableAutoConfiguration. Эта аннотация сообщает **Spring Boot**, что необходимо "угадать", как нужно сконфигурировать Spring, основываясь на добавленных вами jar-зависимостях. Поскольку **spring-boot-starter-web** добавил Tomcat и Spring MVC, средство автоконфигурирования предполагает, что вы разрабатываете веб-приложение, и настраивает Spring соответствующим образом.

AutoConfiguration

Если вы используете **spring-boot-starter-jdbc**, Spring Boot автоматически регистрирует бины **DataSource**, **EntityManagerFactory**, **TransactionManager** и считывает информацию для подключения к базе данных из файла **application.properties**.

Если вы не собираетесь использовать базу данных, и не предоставляете никаких подробных сведений о подключении в ручном режиме, Spring Boot автоматически настроит базу в памяти, без какой-либо дополнительной конфигурации с вашей стороны (при наличии H2 или HSQL Driver'ов).

Автоматическая конфигурация может быть полностью переопределена в любой момент с помощью пользовательских настроек.

demo ./mvnw spring-boot:run --quiet



```
2020-02-14 16:16:47.746 INFO 4838 --- [main] com.example.demo.DemoApplication : Starting DemoApplication
on Brians-MacBook-Pro.local with PID 4838 (/Users/bclozel/workspace/tmp/demo/target/classes started by bclozel in /Users/bclozel/workspace/tmp/demo)
2020-02-14 16:16:47.748 INFO 4838 --- [main] com.example.demo.DemoApplication : No active profile set, falling back to default profiles: default
2020-02-14 16:16:48.272 INFO 4838 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2020-02-14 16:16:48.279 INFO 4838 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2020-02-14 16:16:48.279 INFO 4838 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.30]
2020-02-14 16:16:48.323 INFO 4838 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2020-02-14 16:16:48.324 INFO 4838 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 532 ms
2020-02-14 16:16:48.438 INFO 4838 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2020-02-14 16:16:48.533 INFO 4838 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2020-02-14 16:16:48.535 INFO 4838 --- [main] com.example.demo.DemoApplication : Started DemoApplication in 1.006 seconds (JVM running for 1.248)
```

Spring Initializr

Чтобы каждый раз не создавать с нуля **Spring** проект на **Java** и не искать последние версии зависимостей, можно воспользоваться сервисом [Spring Initializr](#), который предоставляет интерфейс для генерации заготовки проекта с добавлением стандартных зависимостей. Их можно конфигурировать в зависимости от ваших потребностей.

В качестве сборщика проекта выберем предпочитаемый, и **Spring Initializr** автоматически сгенерирует скрипт сборки.

В **IntelliJ Idea** от **JetBrains** на данный момент уже имеет интеграцию с этим генератором, можно создавать шаблон непосредственно из IDE.



spring initial

Web, Security, JPA, Actuator, Devtools...

Press Ctrl for multiple adds

DEVELOPER TOOLS

GraalVM Native Support



Spring Boot DevTools

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Lombok

Java annotation library which helps to reduce boilerplate code.

Spring Configuration Processor

Generate metadata for developers to offer contextual help and "code completion" when working with custom configuration keys (ex.application.properties/.yml files).

WEB

Spring Web

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Reactive Web

Build reactive web applications with Spring WebFlux and Netty.

[illegible]

Project

☐ Gradle - Groovy

● Gradle - Kotlin ○ Maven

Spring Boot

☐ 3.1.0 (SNAPSHOT) ☐ 3.1.0 (M

☐ 2.7.11 (SNAPSHOT) ☐ 2.7.10

Project Metadata





demo.zip

- .gitignore
- HELP.md
- build.gradle.kts
- gradle
 - gradlew
 - gradlew.bat
 - settings.gradle.kts
- src
 - main
 - java
 - com
 - example
 - demo
 - DemoApplication.java
 - resources

DOWNLOAD

COPY

```
16
17 repositories {
18     mavenCentral()
19 }
20
21 dependencies {
22     implementation("org.springframework.boot:spring-boot-starter-data-jpa")
23     implementation("org.springframework.boot:spring-boot-starter-web")
24     implementation("org.flywaydb:flyway-core")
25     compileOnly("org.projectlombok:lombok")
26     developmentOnly("org.springframework.boot:spring-boot-devtools")
27     runtimeOnly("com.h2database:h2")
28     runtimeOnly("org.postgresql:postgresql")
29     annotationProcessor("org.springframework.boot:spring-boot-configuration-processor")
30     annotationProcessor("org.projectlombok:lombok")
31     testImplementation("org.springframework.boot:spring-boot-starter-test")
32 }
33
34 tasks.withType<Test> {
35     useJUnitPlatform()
36 }
37
```



DOWNLOAD CTRL + ⌘

CLOSE ESC

Автоконфигурация

Автоконфигурация в **Spring Boot** пытается автоматически конфигурировать ваше приложение **Spring** на основе добавленных jar-зависимостей. Например, если HSQLDB находится в вашем classpath, но вы не сконфигурировали вручную никаких бинов для подключения к базе данных, то Spring Boot автоматически сконфигурирует резидентную базу данных.

Необходимо явно согласиться на автоконфигурацию, добавив аннотации **@EnableAutoConfiguration** или **@SpringBootApplication** в один из ваших классов с аннотацией **@Configuration**.

Постепенная замена автоконфигурации

Автоконфигурация работает неагрессивно. В любой момент можно начать определять свою собственную конфигурацию, чтобы заменить определенные части автоконфигурации. Например, если вы добавите свой собственный бин **DataSource**, то средства поддержки встроенной базы данных по умолчанию отключатся.

Если необходимо узнать, какая автоконфигурация применяется в данный момент и почему, запустите приложение с параметром **--debug**. Это позволит активировать отладочные журналы для выбранных основных диспетчеров журналирования и вывести отчет об условиях на консоль.

Отключение автоконфигурации

Если вы обнаружите, что применяются определенные классы автоконфигурации, которые вам не нужны, то можете использовать атрибут исключения (exclude) в аннотации @SpringBootApplication

Java

```
1  import org.springframework.boot.autoconfigure.SpringBootApplication;
2  import org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration;
3  @SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
4  public class MyApplication {
5  }
```

Spring Beans и внедрение зависимостей

Когда вы будете структурировать свой код, то вам понадобится добавлять аннотацию `@ComponentScan` или использовать аннотацию `@SpringBootApplication`, которая неявно содержит её. Все компоненты вашего приложения (аннотации `@Component`, `@Service`, `@Repository`, `@Controller` и другие) автоматически регистрируются как бины Spring.

Рассмотрим на примере.

```
12    })
13    @ComponentScan(basePackages = {
14
15        Choose Bean
16        application (controller\Application.java)
17        catController (CatController.java)
18        catRepository (CatRepository.java)
19        catServiceImpl (CatServiceImpl.java)
20    })
21 }
```

Spring: Beans MVC Data

is-y25-tech.k	/\${server.error.pat	/api/cat [POST]
is-y25-tech.k	/api/cat/* (com.m	/api/cat/search [GET]
is-y25-tech.k	graphQLSocketCc	/api/cat/{id} [GET]
is-y25-tech.k	/api/owner/* (cor	/api/cat/{id}/friend [POST]
is-y25-tech.k	swaggerUiHome (c	
is-y25-tech.k	swaggerWelcome	

Git TODO Problems Spring Terminal Services Pro

Инструменты разработчика

Spring Boot содержит дополнительный набор инструментальных средств, которые могут сделать процесс разработки приложений немного приятнее. Модуль **spring-boot-devtools** можно добавлять в любой проект для обеспечения дополнительных функций во время разработки.

Инструменты разработчика

Инструментальные средства разработки автоматически деактивируются при выполнении полностью упакованного приложения. Если ваше приложение запускается через `java -jar` или через специальный загрузчик классов, то оно считается **Production**-сборкой.

Вы можете управлять этой логикой работы при помощи системного свойства **`spring.devtools.restart.enabled`**. Однако нужно понимать что это нельзя делать в Production среде, где выполнение devtools представляет угрозу безопасности.

Автоматический перезапуск

Приложения, использующие **spring-boot-devtools**, автоматически перезапускаются при изменении файлов в classpath. Это может быть полезной функцией при работе в IDE, так как обеспечивает очень быструю обратную связь для внесения изменений в код. По умолчанию любая запись в classpath, указывающая на каталог, отслеживается на предмет изменений. Обратите внимание, что некоторые ресурсы, такие как статическое содержимое и шаблоны представлений, не требуют перезапуска приложения.

Данный подход в индустрии также известен как **Hot Module Reload**

Перезапуск и перезагрузка

Технология перезапуска, предусмотренная Spring Boot, работает при помощи двух загрузчиков классов. Классы, которые не изменяются (например, классы из сторонних jar-файлов), загружаются в *основной* загрузчик классов. Классы, которые активно разрабатываются, загружаются в *перезапускающий* загрузчик классов. Если приложение перезапускается, *перезапускающий* загрузчик классов единовременно используется, после чего создается новый. Такой подход означает, что перезапуск приложения обычно происходит гораздо быстрее, чем "холодный запуск", поскольку *основной* загрузчик классов уже доступен и заполнен.

Цикл жизни Spring Application

В дополнение к обычным событиям Spring Framework, таким как **ContextRefreshedEvent**, на которое в основном и реагирует dev-tools для обеспечения горячей замены разрабатываемых Bean'ов, Spring посылает некоторые дополнительные события.

События приложения отправляются в следующем порядке по мере выполнения приложения:

1. Событие **ApplicationStartingEvent** отправляется в начале выполнения, но перед началом любой обработки, за исключением регистрации слушателей и инициализаторов.

Цикл жизни Spring Application

2. Событие **ApplicationEnvironmentPreparedEvent** отправляется, когда **Environment**, которое будет использоваться в контексте, известно, но перед созданием контекста.

3. Событие **ApplicationContextInitializedEvent** отправляется, когда **ApplicationContext** подготовлен и вызваны **ApplicationContextInitializers**, но перед загрузкой определений бинов.

4. Событие **ApplicationPreparedEvent** отправляется непосредственно перед началом обновления, но после загрузки определений бинов.

Цикл жизни Spring Application

5. Событие **ApplicationStartedEvent** отправляется после обновления контекста, но перед тем, как будут вызваны все средства выполнения приложения и командной строки.
6. Сразу после этого отправляется событие **AvailabilityChangeEvent** с *LivenessState.CORRECT*, чтобы обозначить, что приложение считается работающим.
7. Событие **ApplicationReadyEvent** отправляется после вызова любого средства выполнения приложений и командной строки.

Цикл жизни Spring Application

8. Сразу после этого отправляется событие **AvailabilityChangeEvent** с *ReadinessState.ACCEPTING_TRAFFIC*, чтобы обозначить, что приложение готово к обработке запросов.

9. Событие **ApplicationFailedEvent** отправляется, если при запуске возникло исключение.

Приведенный выше список включает только события **SpringApplicationEvent**, которые привязаны к **SpringApplication**.

Дополнительные события

В дополнение к ним следующие события также публикуются после **ApplicationPreparedEvent** и перед **ApplicationStartedEvent**:

- Событие **WebServerInitializedEvent** отправляется после готовности **WebServer**. **ServletWebServerInitializedEvent** и **ReactiveWebServerInitializedEvent** – это варианты сервлета и реактивного сервера соответственно.
- Событие **ContextRefreshedEvent** отправляется, если обновляется **ApplicationContext**.



Зачастую нет нужды использовать события приложения, но знать об их существовании может быть полезно. На внутреннем уровне Spring Boot события используются для выполнения различных задач.



Слушатели событий не должны выполнять потенциально длительные задачи, поскольку по умолчанию они выполняются в одном потоке. Рассмотрите возможность использования средств выполнения приложения и командной строки вместо этого.

Рассмотрим как это делать на примере
(может пригодится когда вам нужно будет применять миграции)

Java

```
1  import java.util.List;
2  import org.springframework.boot.ApplicationArguments;
3  import org.springframework.stereotype.Component;
4  @Component
5  public class MyBean {
6      public MyBean(ApplicationArguments args) {
7          boolean debug = args.containsOption("debug");
8          List<String> files = args.getNonOptionArgs();
9          if (debug) {
10             System.out.println(files);
11         }
12         // если запустить с параметром "--debug logfile.txt", выводится ["logfile.txt"].
13     }
14 }
```

Тестирование приложения

Разработка Spring приложения с использованием dev-tools это безусловно удобно, но постоянно вызывать один и тот же метод для отладки может быть утомительным и в конечном счете появляется потребность в автоматизации данных действий.

Хотелось бы иметь возможность использовать уже привычным нам JUnit для того чтобы проверять корректность работы нашей бизнес логики, но без обязательного развёртывания приложения.

Выход есть...

Тестирование приложения

Spring Boot предусматривает ряд утилит и аннотаций, упрощающих тестирование приложения. Поддержка тестирования обеспечивается двумя модулями: **spring-boot-test** содержит основные элементы, а **spring-boot-test-autoconfigure** поддерживает автоконфигурацию для тестов.

Большинство разработчиков используют "стартер" **spring-boot-starter-test**, который импортирует оба тестовых модуля Spring Boot, а также JUnit Jupiter, AssertJ, Hamcrest и ряд других полезных библиотек.

Что внутри spring-boot-starter-test ?

JUnit 5: Стандарт де-факто для модульного тестирования Java-приложений.

Spring Test и Spring Boot Test: Средства поддержки утилит и интеграционных тестов для приложений Spring Boot.

AssertJ: Библиотека текущих утверждений.

Hamcrest: Библиотека объектов-сопоставителей (matchers) (также известных как ограничения или предикаты).

Mockito: Java-фреймворк для мокирования объектов

JSONassert: Библиотека утверждений для JSON.

JsonPath: XPath для JSON.

Тестирование приложений Spring

Одно из главных преимуществ внедрения зависимостей заключается в том, что оно должно облегчить модульное тестирование кода. Можно создавать экземпляры объектов с помощью оператора `new`, даже не вовлекая Spring. Также можно использовать объекты-имитации (Mock) вместо реальных зависимостей.

Тестирование приложений Spring

Приложение Spring Boot – это **ApplicationContext** для Spring, поэтому для его тестирования не требуется ничего особенного, кроме тех операций, которые выполняются для ванильного контекста Spring.

Spring Boot предусматривает аннотацию **@SpringBootTest**, которую по сути используется в качестве альтернативы стандартной аннотации **@ContextConfiguration**

Тестирование приложений Spring

Если Spring MVC доступен, конфигурируется обычный контекст приложения на основе MVC.

При тестировании приложений Spring Boot это обычно не требуется. Аннотации **@*Test** в Spring Boot осуществляют поиск вашей первичной конфигурации автоматически, если она не была определена вами явно. Алгоритм поиска начинает работу с пакета, содержащего тест, пока не найдет класс, аннотированный **@SpringBootApplication** или **@SpringBootConfiguration**. При условии, что вы структурировали свой код адекватным образом, основную конфигурацию обычно удастся найти.

Тестирование приложений Spring

По умолчанию аннотация **@SpringBootTest** не запускает сервер, а вместо этого создает имитационное окружение для тестирования конечных веб-точек (т.е. ваших Endpoint'ов).

В Spring MVC можно запрашивать конечные веб-точки с помощью специального объекта – **MockMvc**.

Рассмотрим на примере.

```
31 @SpringBootTest
32 @AutoConfigureMockMvc
33 public class CatControllerTest {
34
35     @Autowired
36     protected MockMvc mockMvc;
37
38     @Test
39     void findOwnerByIdOk() throws Exception {
40         String resultJson = mockMvc
41             .perform(get(urlTemplate: "/api/owner/1")) ResultActions
42             .andExpect(status().isOk())
43             .andReturn() MvcResult
44             .getResponse() MockHttpServletResponse
45             .getContentAsString();
46
47         JSONAssert.assertEquals(expectedStr: "{id: 1, name: \"Roma\"}",
48             resultJson,
49             JSONCompareMode.NON_EXTENSIBLE
50         );
51     }
```

Тестирование приложений Spring

В примере то всё хорошо, но...

Если мы использовали имитацию работающего приложения, соответственно и база данных у нас это тоже временно поднятая в памяти H2 (по умолчанию).

Как там появятся необходимые данные, которые мы проверяем в тесте?

Давайте разбираться, естественно тоже на примере.

<https://github.dev/springtestdbunit/spring-test-dbunit/tree/master/spring-test-dbunit-sample>